

```

import matplotlib.pyplot as plt
import math

def resolver_gauss(A, b):
    n = len(b)
    for i in range(n):
        # Pivoteamento parcial
        max_el = abs(A[i][i])
        max_row = i
        for k in range(i + 1, n):
            if abs(A[k][i]) > max_el:
                max_el = abs(A[k][i])
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        # Eliminação
        for k in range(i + 1, n):
            c = -A[k][i] / A[i][i]
            for j in range(i, n):
                A[k][j] += c * A[i][j]
            b[k] += c * b[i]

        # Substituição retroativa
        x = [0 for _ in range(n)]
        for i in range(n - 1, -1, -1):
            soma = sum(A[i][j] * x[j] for j in range(i + 1, n))
            x[i] = (b[i] - soma) / A[i][i]
        return x

# -----
# 1. ENTRADA DE DADOS INTERATIVA
# -----

print("--- Simulador de Treliças Isostáticas ---")
N = int(input("Número de nós: "))
x, y = [], []
for i in range(N):
    coords = input(f"Coordenadas do Nó {i} (x y): ").replace(',', ' ')
    x.append(float(coords[0]))
    y.append(float(coords[1]))

NB = int(input("Número de barras: "))

```

```

barras = []
for i in range(NB):
    conn = input(f"Conectividade da Barra {i} (nó_inicial nó_final): ")
    conn = conn.replace(',', ' ').split()
    barras.append((int(conn[0]), int(conn[1])))

Fx, Fy = [0.0]*N, [0.0]*N
print("\n--- Cargas Aplicadas (digite 0 se não houver) ---")
for i in range(N):
    cargas = input(f"Cargas no Nó {i} (Fx Fy): ").replace(',', ' ')
    cargas = cargas.split()
    Fx[i] = float(cargas[0])
    Fy[i] = float(cargas[1])

apoios = []
print("\n--- Tipos de Apoio: 0=Livre, 1=Móvel(Y), 2=Fixo(XY) ---")
for i in range(N):
    apoios.append(int(input(f"Tipo de apoio no Nó {i}: ")))

# -----
# 2. MONTAGEM E CÁLCULO
# -----

reacoes_map = []
for i, tipo in enumerate(apoios):
    if tipo == 2: # Fixo
        reacoes_map.append((i, 'x'))
        reacoes_map.append((i, 'y'))
    elif tipo == 1: # Móvel (reage apenas em Y)
        reacoes_map.append((i, 'y'))

total_incognitas = NB + len(reacoes_map)
dim = 2 * N

if total_incognitas != dim:
    print(f"\nERRO: Sistema Hipostático ou Hiperestático  
( {total_incognitas} incógnitas vs {dim} equações).")
else:
    A = [[0.0 for _ in range(total_incognitas)] for _ in range(dim)]
    b = [0.0 for _ in range(dim)]

    for i in range(N):
        b[2*i] = -Fx[i]
        b[2*i+1] = -Fy[i]

```

```

for j, (n1, n2) in enumerate(barras):
    if i == n1 or i == n2:
        dx, dy = x[n2]-x[n1], y[n2]-y[n1]
        L = math.sqrt(dx**2 + dy**2)
        sentido = 1 if i == n1 else -1
        A[2*i][j] = sentido * (dx/L)
        A[2*i+1][j] = sentido * (dy/L)

for r_idx, (no_r, direcao) in enumerate(reacoes_map):
    if i == no_r:
        col = NB + r_idx
        if direcao == 'x': A[2*i][col] = 1.0
        else: A[2*i+1][col] = 1.0

sol = resolver_gauss(A, b)
f_barras = sol[:NB]

# -----
# 3. RESULTADOS E GRÁFICOS
# -----
print("\n--- RESULTADOS ---")
for i, f in enumerate(f_barras):
    status = "Tração (+)" if f > 1e-6 else "Compressão (-)" if f <
-1e-6 else "Nula"
    print(f"Barra {i}: {f:.2f} | {status}")

fig, axs = plt.subplots(2, 2, figsize=(12, 8))

# Treliça Original
for i, (n1, n2) in enumerate(barras):
    axs[0,0].plot([x[n1], x[n2]], [y[n1], y[n2]], 'k-o', alpha=0.5)
    axs[0,0].text((x[n1]+x[n2])/2, (y[n1]+y[n2])/2, f'B{i}',
color='red')
axs[0,0].set_title("Geometria Original")

# Deformada/Calor (Simulação visual de esforço)
norm = plt.Normalize(min(f_barras)-1, max(f_barras)+1)
cmap = plt.get_cmap('coolwarm')
for i, (n1, n2) in enumerate(barras):
    cor = cmap(norm(f_barras[i]))
    axs[0,1].plot([x[n1], x[n2]], [y[n1], y[n2]], color=cor, lw=3)
axs[0,1].set_title("Esforços (Azul: Tração | Vermelho: Comp.)")

```

```

# Gráfico de Barras
axs[1,0].bar([f'B{i}' for i in range(NB)], f_barras, color='gray')
axs[1,0].set_title("Magnitude das Forças")

# Histograma
axs[1,1].hist(f_barras, bins=5, color='orange', edgecolor='black')
axs[1,1].set_title("Distribuição de Forças")

plt.tight_layout()
plt.show()

```

```

--- Simulador de Treliças Isostáticas ---
Número de nós: 4
Coordenadas do Nó 0 (x y): 0, 0
Coordenadas do Nó 1 (x y): 4, 0
Coordenadas do Nó 2 (x y): 2, 2
Coordenadas do Nó 3 (x y): 2, 0
Número de barras: 5
Conectividade da Barra 0 (nó_inicial nó_final): 0, 2
Conectividade da Barra 1 (nó_inicial nó_final): 2, 1
Conectividade da Barra 2 (nó_inicial nó_final): 0, 3
Conectividade da Barra 3 (nó_inicial nó_final): 3, 1
Conectividade da Barra 4 (nó_inicial nó_final): 2, 3

--- Cargas Aplicadas (digite 0 se não houver) ---
Cargas no Nó 0 (Fx Fy): 0, 0
Cargas no Nó 1 (Fx Fy): 0, 0
Cargas no Nó 2 (Fx Fy): 0, 0
Cargas no Nó 3 (Fx Fy): 0, -1000

--- Tipos de Apoio: 0=Livre, 1=Móvel(Y), 2=Fixo(XY) ---
Tipo de apoio no Nó 0: 2
Tipo de apoio no Nó 1: 1
Tipo de apoio no Nó 2: 0
Tipo de apoio no Nó 3: 0

--- RESULTADOS ---
Barra 0: -707.11 | Compressão (-)
Barra 1: -707.11 | Compressão (-)
Barra 2: 500.00 | Tração (+)
Barra 3: 500.00 | Tração (+)
Barra 4: 1000.00 | Tração (+)

```

